Solution: Beautiful Array

Problem Statement

Given a positive integer `N`, construct a beautiful array of size `N` containing the number `[1, N]` .

An array is beneficial if it follows the conditions below.

- If for any three indices `i`, `j,` `k` (with `i < j < k`), `A[j] * 2` is not equal to `A[i] + A[k].`

Examples

Example 1

- **Input**: `4`
- **Expected Output**: `[2, 1, 4, 3]`
- **Justification**: In this array, no combination of `i`, `j`, `k` (where `i < j < k`) exists such that `2 * A[j] = A[i] + A[k]`.

Example 2

- **Input**: `3`
- **Expected Output**: `[1, 3, 2]`
- **Justification**: Similar to example 1, this array also satisfies the condition for a smaller size.

Example 3

- **Input**: `8`
- **Expected Output**: `[1, 5, 3, 7, 2, 6, 4, 8]` (or any other permutation that satisfies the condition)
- **Justification**: In this array, for every `i, j, k` (where `i < j < k`), `2 * A[j]` is never equal to `A[i] + A[k]`. This output ensures that all integers from 1 to 8 are used, and the condition is met for all possible triplets.

Solution

To solve the problem, a divide-and-conquer strategy, which focuses on the inherent properties of odd and even numbers, can be used. The fundamental approach is to construct two separate arrays, one consisting entirely of odd numbers and the other of even numbers. The reason for this segregation is rooted in the basic arithmetic property that the sum of an odd and an even number is always odd. Therefore, by ensuring one array contains only odds and the other only evens, we prevent the formation of any triplet `i, j, k` (with `i < j < k`) for which `A[j] * 2 = A[i] + A[k]`.

To recursively generate these subarrays for smaller sizes, create two smaller beautiful arrays - one for `N/2` (even elements) and the other for `(N+1)/2` (odd elements). In this recursive process, the base case occurs when `N` equals 1, yielding a single-element array `[1]`.

To merge these small subarrays, transform the elements of the odd array by multiplying each by 2 and subtracting 1, thereby preserving their odd nature. Simultaneously, elements of the even array are doubled, maintaining their even status. This transformation is key to maintaining the distinctiveness of the odd and even arrays. Finally, these transformed arrays are concatenated. This final step combines the odd and even elements in such a way that the resulting array maintains the 'beautiful' property for the entire set, adhering to the condition that no triplet of indices `i, j, k` (with `i < j < k`) in the array can satisfy `A[j] * 2 = A[i] + A[k]`.

Step-by-step algorithm

1. **Divide the Problem**: We divide the problem into creating two smaller 'beautiful' arrays - one for odd and one for even integers. This division is based on the insight that a sum of an odd and an even number is always odd, and thus can never be twice any number in the array.
2. **Recursive Construction**: We recursively construct these smaller arrays. For a given size `N`, we create beautiful arrays for `N/2` (even numbers) and `(N+1)/2` (odd numbers). During recursion, if `N` becomes 1, we return `[1]` as the base case.
3. **Transforming Subarrays**: After obtaining the smaller arrays, we transform them to fit into the larger structure. For the odd array, we multiply each element by 2 and subtract 1. For the even array, we simply multiply each element by 2. This step ensures that the elements in the odd array remain odd and those in the even array remain even.
4. **Merging Arrays**: Finally, we merge the transformed odd and even arrays. This merging is done by concatenating the arrays, as they are already structured to maintain the 'beautiful' property.

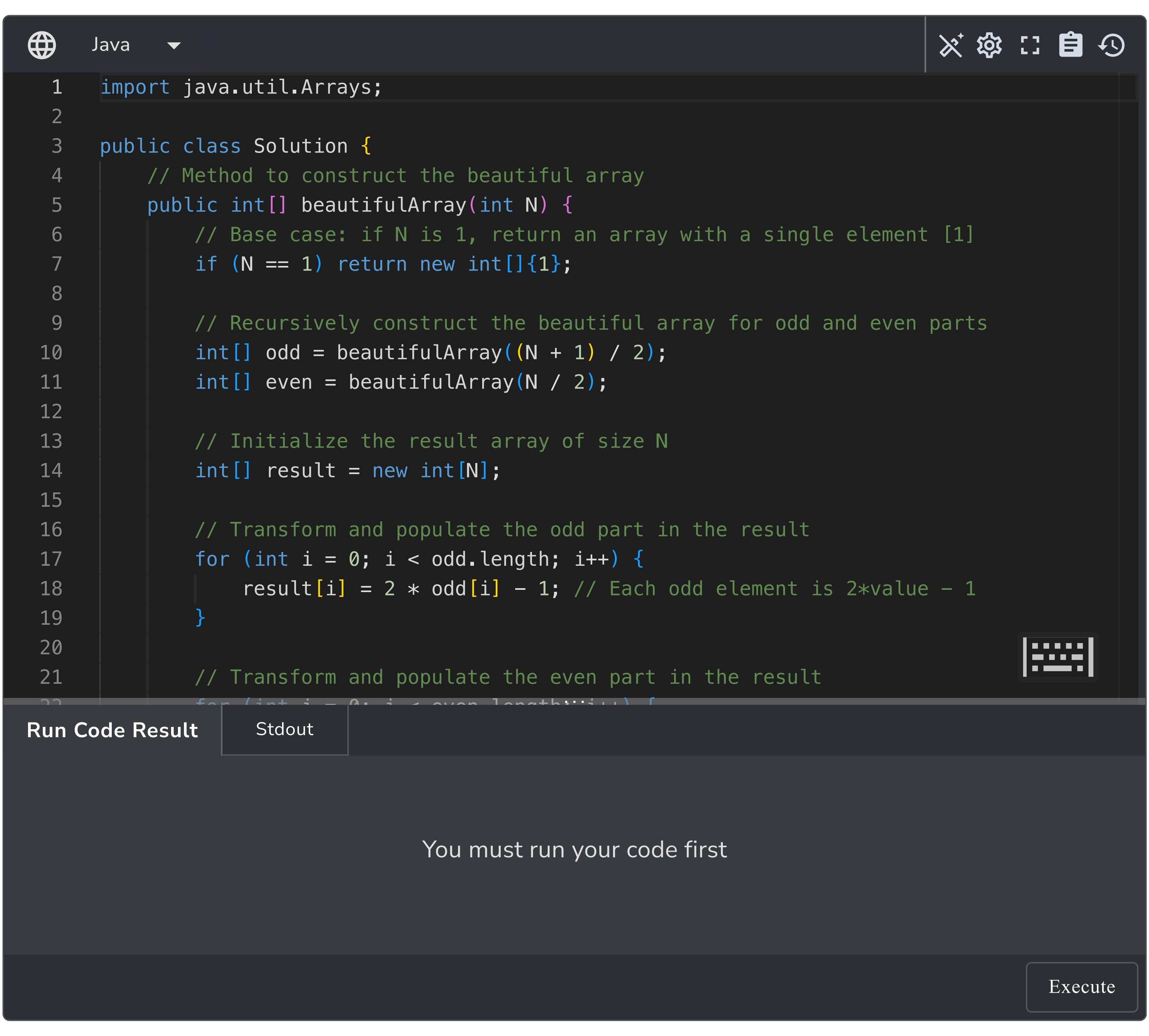
Algorithm Walkthrough

For input `N = 8`, the algorithm functions as follows:

1. **Initial Division**: We start by dividing the problem into two subproblems - creating a beautiful array for the odd numbers and another for the even numbers. For `N = 8`, we need to create beautiful arrays for `4` (odd numbers) and `4` (even numbers), as `8/2 = 4` and `(8+1)/2 = 4`.
2. **Recursive Calls for Odds and Evens**:
    - For the odd part (`N = 4`), we recursively call the algorithm. Since `4` is not the base case, we further divide it into `2` (odds) and `2` (evens).
    - Similarly, for the even part (`N = 4`), we again call the algorithm recursively and divide it into `2` (odds) and `2` (evens).
3. **Base Case Resolution**:
    - For `N = 2`, the recursive calls reach the base case. The beautiful array for `N = 1` is `[1]`.
    - We then transform these base arrays for odds and evens. For odds, `[1]` becomes `[1*2-1] = [1]`, and for evens, `[1]` becomes `[1*2] = [2]`.
    - Combining these for each `N = 2` scenario, we get `[1, 2]`.
4. **Building Up from Base Case**:
    - We now return to the previous recursive step (for `N = 4`). We transform the smaller odd and even arrays (`[1, 2]`) to fit into our larger structure. For odds, `[1, 2]` becomes `[1*2-1, 2*2-1] = [1, 3]` and for evens `[1*2, 2*2] = [2, 4]`.
    - We then combine these to form `[1, 3, 2, 4]` for each `N = 4` scenario.
5. **Final Combination**:
    - Finally, for `N = 8`, we merge the two `N = 4` arrays. For the odd array `[1, 3, 2, 4]`, we transform it to `[1*2-1, 3*2-1, 2*2-1, 4*2-1] = [1, 5, 3, 7]`. For the even array `[1, 3, 2, 4]`, we transform it to `[1*2, 3*2, 2*2, 4*2] = [2, 6, 4, 8]`.
    - Combining these arrays, we get `[1, 5, 3, 7, 2, 6, 4, 8]`.

Code

Here is the code for this algorithm:



```java
import java.util.Arrays;

public class Solution {
    // Method to construct the beautiful array
    public int[] beautifulArray(int N) {
        // Base case: if N is 1, return an array with a single element [1]
        if (N == 1) return new int[]{1};

        // Recursively construct the beautiful array for odd and even parts
        int[] odd = beautifulArray((N + 1) / 2);
        int[] even = beautifulArray(N / 2);

        // Initialize the result array of size N
        int[] result = new int[N];

        // Transform and populate the odd part in the result
        for (int i = 0; i < odd.length; i++) {
            result[i] = 2 * odd[i] - 1; // Each odd element is 2*value - 1
        }

        // Transform and populate the even part in the result
```

You must run your code first

Time and Space Complexity Analysis

- **Time Complexity**: The time complexity of the algorithm is `O(N log N)`. This arises from the recursive division of the problem into two halves at each step (sizes N/2 and (N+1)/2), leading to `log N` recursive levels. At each level, operations proportional to the size of the subproblem are performed. Hence, the combination of linear operations at each logarithmic level results in `O(N log N)` complexity.
- **Space Complexity**: The space complexity of the algorithm is also `O(N log N)`. This is primarily due to the recursive nature of the algorithm, which requires additional space for the stack frames used in the recursive calls.

← Previous
Beautiful Array (Medium)
Next →
Median Of Two Sorted Arrays (Medium)
☐ Mark as Completed